Group 9 - Toxic comments classification

Reyhaneh Kharazmi Pour, Helia Mirsoofian, and Iman Mohammadi

November 2025

1 Abstract

This project explores the classification of toxic online comments using the **Jigsaw dataset** from **Kaggle**. We investigate a variety of machine learning approaches, ranging from classical linear models with **TF-IDF** features to **deep learning architectures** and **gradient boosting**. Our analysis reveals that while **simple Logistic Regression** is a formidable baseline (AUC 0.978), tree-based models like **Random Forest** struggle with high-dimensional sparse text. We implemented a **1D Convolutional Neural Network (CNN)** and **LightGBM** to capture non-linear patterns. Ultimately, we demonstrate that a heterogeneous ensemble of **Logistic Regression** and **CNN** achieves the highest performance with a mean ROC-AUC of **0.9816**. Furthermore, we address the critical issue of class imbalance not just through metrics, but by mathematically optimizing classification thresholds, significantly improving the detection of rare classes like threats.

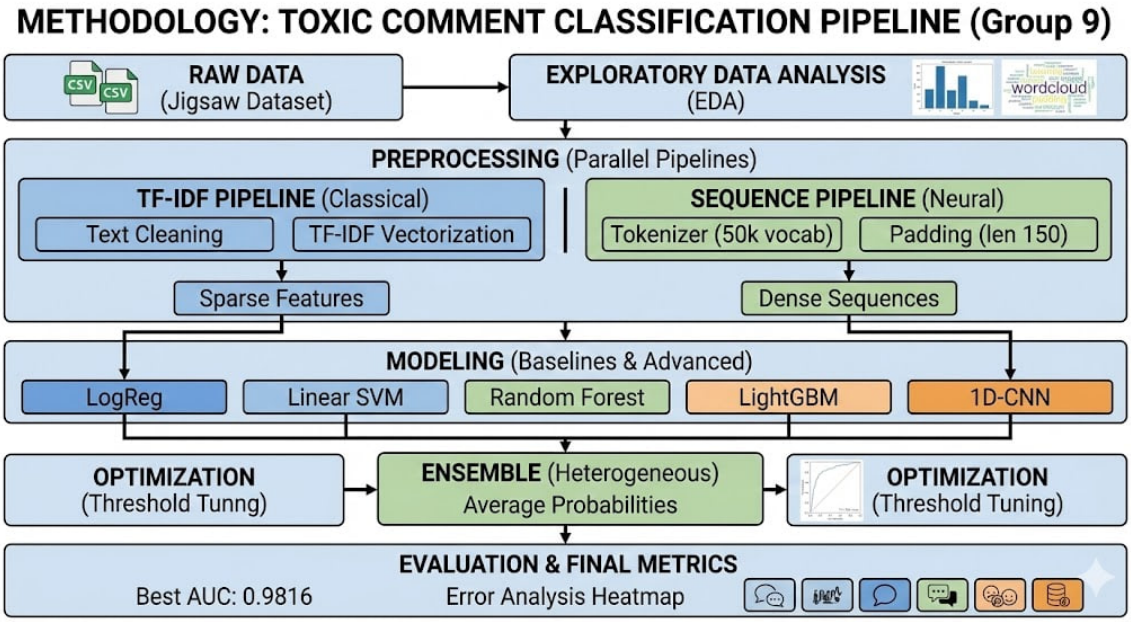

Figure 1: The project methodology.

2 Introduction

Online platforms struggle to effectively facilitate conversations due to the threat of abuse and harassment. The goal of this project is to build a multi-headed model capable of detecting different types of toxicity: `toxic`, `severe_toxic`, `obscene`, `threat`, `insult`, and `identity_hate`. The challenge lies not only in the linguistic nuance of hate speech but also in the severe class imbalance inherent in the data.

3 Dataset Exploration

Class Imbalance. An important characteristic of the Toxic Comment dataset is its severe class imbalance. As shown in Figure 2, only 9.6% of comments are labeled as `toxic`, while categories such as `threat` (0.3%) and `identity_hate` (0.9%) are extremely rare. This imbalance has significant implications for model evaluation: a classifier that predicts the negative class for every instance would still achieve approximately 90-99% accuracy across the labels, despite being entirely useless

for detecting harmful content. Consequently, accuracy is not an informative metric for this task, and we rely instead on class-sensitive metrics such as ROC-AUC and per-label performance. The imbalance also motivates the use of techniques such as class weighting and oversampling to improve minority-label detection.

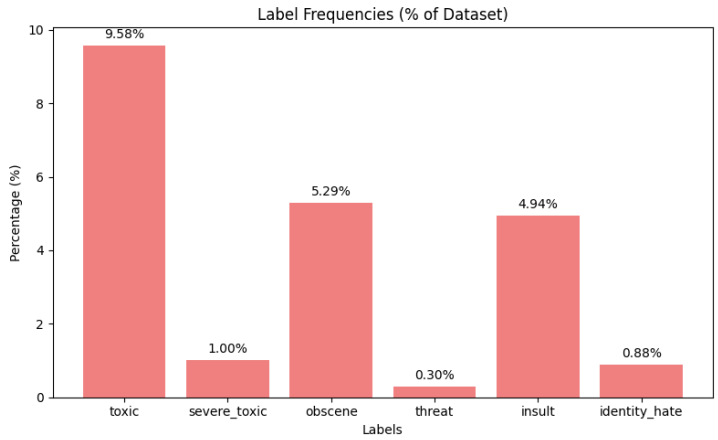

Figure 2: Label frequency distribution of the dataset.

Comment Length Analysis. To better understand linguistic patterns associated with toxic behavior, we examined the distribution of comment lengths using both boxplots and density-based histograms for all six toxicity labels. Across every label, a consistent pattern emerges: comments containing toxic content are substantially *shorter* than non-toxic comments. In the boxplots, the "1 (present)" group shows a noticeably lower median and a much narrower interquartile range compared to the "0 (absent)" group, indicating that toxic, obscene, insulting, and hate-filled comments tend to be brief and direct. The density histograms reinforce this result: toxic comments peak sharply at approximately 5-15 words, while non-toxic comments display a wider distribution with a long right tail extending to 70-80 words. This reflects typical behavior on Wikipedia talk pages, where civil discussions often appear as long explanations or reasoned arguments, whereas toxic comments are often short, emotional outbursts.

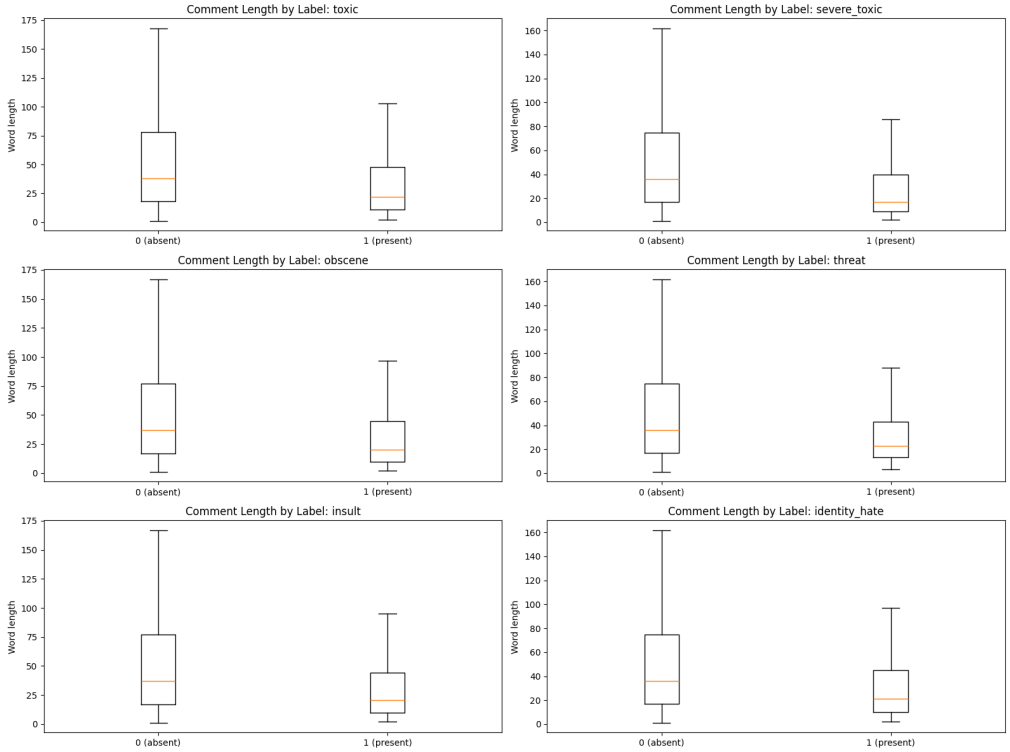

Figure 3: Boxplots of comment word length for each toxicity label, comparing comments where the label is absent (0) versus present (1).

Helia Mirsoofian (35623270)
Iman Mohammadi (51478179)
Reyhaneh Kharazmi Pour (28943962)

Label Co-Occurrence Analysis The co-occurrence matrix (Figures **??**-4) shows that the task is inherently *multi-label*. The labels `toxic`, `obscene`, and `insult` form a strong cluster, frequently appearing together; models can exploit these shared lexical patterns. In contrast, `severe_toxic` is almost entirely a subset of `toxic`, indicating that it provides little independent signal. The `threat` label is extremely rare and largely isolated, making it difficult to learn without class-imbalance strategies such as oversampling or class weighting. `identity_hate` shows moderate overlap with `toxic` and `obscene`, suggesting that hate speech typically co-occurs with more general toxicity. Overall, these patterns justify the use of sigmoid outputs and binary cross-entropy loss, and highlight the need for imbalance-aware methods to handle rare categories.

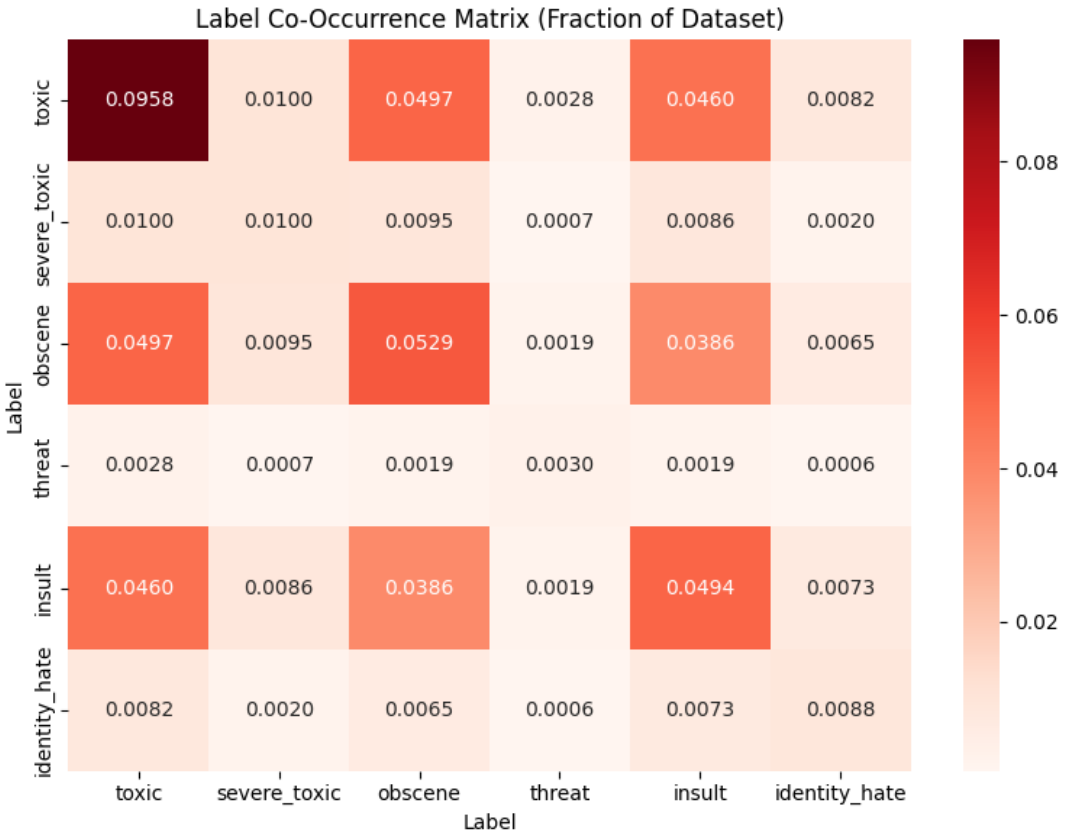

Figure 4: Label co-occurrence matrix (fraction of dataset).

Data Quality Checks. We examined the dataset for potential issues such as empty comments, extremely short messages, punctuation-only entries, emoji-only content, and duplicated comments. The dataset contains no empty or malformed comments, and no instances composed solely of punctuation or emojis. We identified 165 comments containing only one or two words; however, these were retained because short utterances (e.g., "idiot", "stupid", "go away") frequently carry strong toxicity signals and are essential for correct model behavior. No duplicated comment texts were found. Overall, the dataset is clean and requires minimal filtering prior to preprocessing and modeling.

Profanity and Keyword Signals The profanity heatmap reveals that toxicity is strongly driven by a small set of offensive terms. The word `fuck` is the dominant cue across nearly all labels and is especially concentrated in `severe_toxic`, indicating that high-intensity profanity characterizes the strongest forms of abuse. Milder profanity (e.g., `shit`, `fucking`) maps mainly to `obscene` and `toxic`. In contrast, lexical insults such as `idiot`, `stupid`, and `dumb` are most strongly associated with the `insult` label. The term `gay` is highly enriched in `identity_hate`, while `threat` shows very low profanity usage overall, implying that threats rely on structural patterns rather than keywords. These patterns confirm that different toxicity types rely on distinct lexical cues and motivate the use of context-aware models (e.g., transformers) over simple keyword-based methods.

Character-level Patterns Our character-level analysis shows that repeated characters, censored profanity, symbol usage, and mixed-case patterns occur at nearly identical rates across toxic and non-toxic comments. This indicates that character-level distortion is not a meaningful signal in this dataset, and toxicity is driven primarily by word-level features rather than stylization or obfuscation.

Helia Mirsoofian (35623270)
Iman Mohammadi (51478179)
Reyhaneh Kharazmi Pour (28943962)

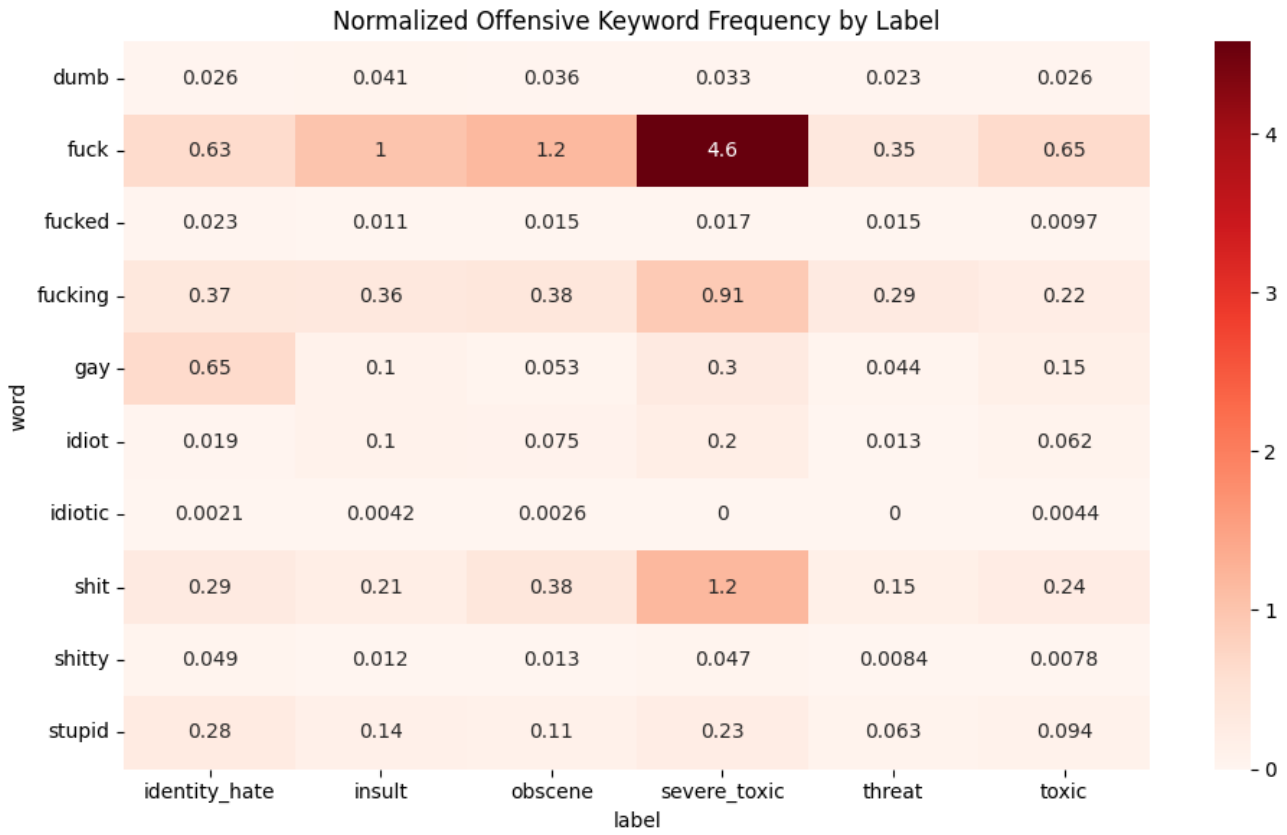

Figure 5: Normalized offensive keyword frequency by label.

Most Frequent Words by Label. Figure 6 summarizes the ten most common unigrams for each toxicity label. Four labels(`toxic`, `severe_toxic`, `obscene`, and `insult`) share a core profanity vocabulary, with words such as `fuck`, `shit`, `suck`, and `bitch` appearing prominently across all of them. This overlap implies that general profanity alone does not distinguish these categories and explains the frequent co-occurrence of these labels in the dataset. In contrast, the `threat` label is dominated by action-oriented verbs (`die`, `kill`, `going`), while `identity_hate` relies on a narrow set of targeted slurs (e.g., `nigger`, `jew`, `gay`). These patterns highlight meaningful semantic differences between toxicity types and motivate the need for context-aware models rather than simple keyword detection.

4 Preprocessing

Although the Wikipedia talk-page dataset is relatively clean, different models required distinct text representations. We implemented two parallel pipelines:

TF-IDF Pipeline (Classical Models). For linear models and trees, we performed aggressive cleaning: lowercasing, removal of extra whitespace, normalization of repeated characters (e.g., "fuuuuuck" → "fuuck"), and correction of censored profanity. These steps help sparse models capture semantically equivalent toxic expressions. We then vectorized the text using TF-IDF with unigrams and bigrams.

Sequence Pipeline (Deep Learning). For the CNN, we retained more of the raw text structure. We used a Keras `Tokenizer` with a vocabulary size of 50,000 words. Sequences were padded to a fixed length of 150 tokens. This preserves local word order, allowing the convolutional layers to detect sequential n-gram patterns.

Finally, we performed a stratified train–validation split (80/20) based on an "any_toxic" flag to preserve the imbalance structure.

5 Modeling Approaches

We explored a progression of complexity from linear models to ensembles.

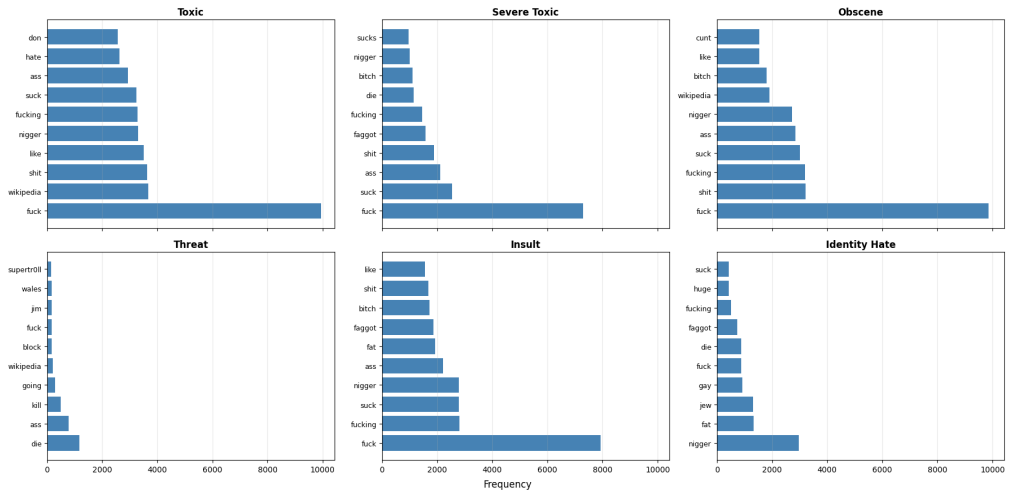

Figure 6: Top 10 most frequent unigrams for each toxicity label.

5.1 Baseline Models

We evaluated three standard multi-label classifiers using TF-IDF features: One-vs-Rest Logistic Regression, Linear SVM, and Random Forests. Table 1 reports the ROC-AUC scores.

Label	LogReg	Linear SVM	Random Forest
toxic	0.9719	0.9653	0.9509
severe_toxic	0.9812	0.9613	0.9607
obscene	0.9862	0.9800	0.9827
threat	0.9794	0.9676	0.9202
insult	0.9789	0.9698	0.9669
identity_hate	0.9708	0.9571	0.9219
Mean	**0.9781**	**0.9668**	**0.9506**

Table 1: ROC-AUC performance of baseline models using TF-IDF features.

Logistic Regression outperformed Random Forest significantly (0.978 vs 0.950). Tree-based models often struggle with high-dimensional sparse data because individual splits capture very little signal when features are sparse.

5.2 Advanced Models

LightGBM (Gradient Boosting). To test if gradient boosting could capture non-linearities better than Random Forest, we implemented LightGBM. While it achieved a respectable mean AUC of **0.9604**, it still trailed the linear models. This reinforces the finding that text classification with TF-IDF is often a linearly separable problem.

Convolutional Neural Network (CNN). We implemented a 1D CNN to capture local n-gram patterns. The architecture consists of an Embedding Layer (Dim: 128), a Conv1D layer (Filters: 128, Kernel: 5), Global Max Pooling, and a Dense Output layer with Sigmoid activation. After hyperparameter tuning, the CNN achieved a mean AUC of **0.9780**. This effectively matches the strong Logistic Regression baseline, proving that a lightweight neural architecture can learn the necessary features from scratch without hand-crafted engineering.

6 Cross-Validation and Model Stability Analysis

To assess the stability and generalization of our baseline models, we conducted a **5-fold cross-validation** experiment on the Logistic Regression classifier using TF-IDF features. Cross-validation provides a more reliable estimate of model performance by training on different partitions of the data, reducing the sensitivity to any single train-test split.

Across the five folds, the model achieved ROC-AUC values ranging from **0.974 to 0.977**. The mean AUC of **0.976**, together with the extremely small standard deviation of **0.0008**, indicates that the model is highly stable and consistently learns similar decision boundaries across folds. This low variance suggests that the TF-IDF representation creates a well-structured feature space in which linear classifiers behave predictably.

Also, the nearly identical bar heights show that performance variation is minimal, demonstrating that the model's predictions are not overly influenced by the particular sample of data used for training. Figure 7 presents the same results in a line plot, highlighting the smooth and steady trend of AUC values across folds. Such stability is an important property for toxicity classification, where reliable behavior across slightly different data samples is essential for real-world deployment. Overall, this analysis confirms that Logistic Regression with TF-IDF features provides a strong and dependable baseline for comparison with more complex models.

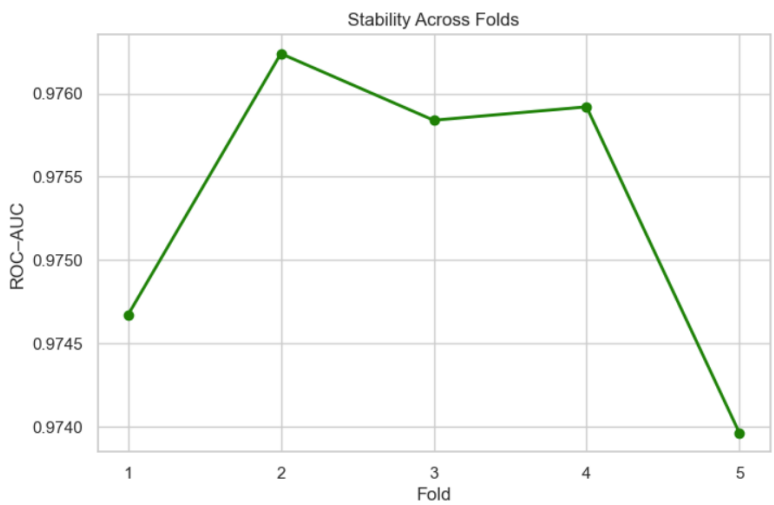

Figure 7: AUC stability curve across folds.

7 Model Selection and Ensembling

Figure 8 summarizes the performance of all models. While Logistic Regression (0.978) and CNN (0.978) performed similarly, they are fundamentally different algorithms. Logistic Regression relies on global keyword occurrences, while CNNs capture local sequential patterns.

To exploit this diversity, we constructed a **Heterogeneous Ensemble** by averaging the probability outputs of the Logistic Regression and the CNN.

$$P_{ensemble} = 0.5 \times P_{LogReg} + 0.5 \times P_{CNN}$$

This ensemble achieved a mean ROC-AUC of **0.9816**, surpassing all individual models. This confirms that combining linear and neural architectures provides a complementary boost in performance, yielding our best overall result.

8 Underfitting and Overfitting Analysis

To evaluate the capacity and generalization behavior of the Logistic Regression baseline, we examined its learning dynamics using a standard learning-curve analysis. The model was trained on increasing fractions of the dataset–from 10% up to 100%–and the ROC-AUC was recorded for both the training and validation sets at each step. Figure 9 summarizes the results.

The training curve remains consistently high (around 0.985), while the validation curve starts lower but increases steadily as more data becomes available. The gap between the two curves is moderate and decreases slightly with larger training sets, indicating that Logistic Regression benefits from additional data but is not overly sensitive to sample size. Importantly, the validation curve does not diverge or plateau prematurely, suggesting no evidence of high-variance overfitting. At the same time, the consistently strong training performance indicates that the model is not capacity-limited, and thus not underfitting.

Overall, the learning curve shows a healthy generalization pattern: Logistic Regression achieves stable performance across training sizes, with gradual improvement in validation AUC as more data is provided. This confirms that the TF-IDF representation combined with a linear classifier forms a robust and well-calibrated baseline for multilabel toxicity classification.

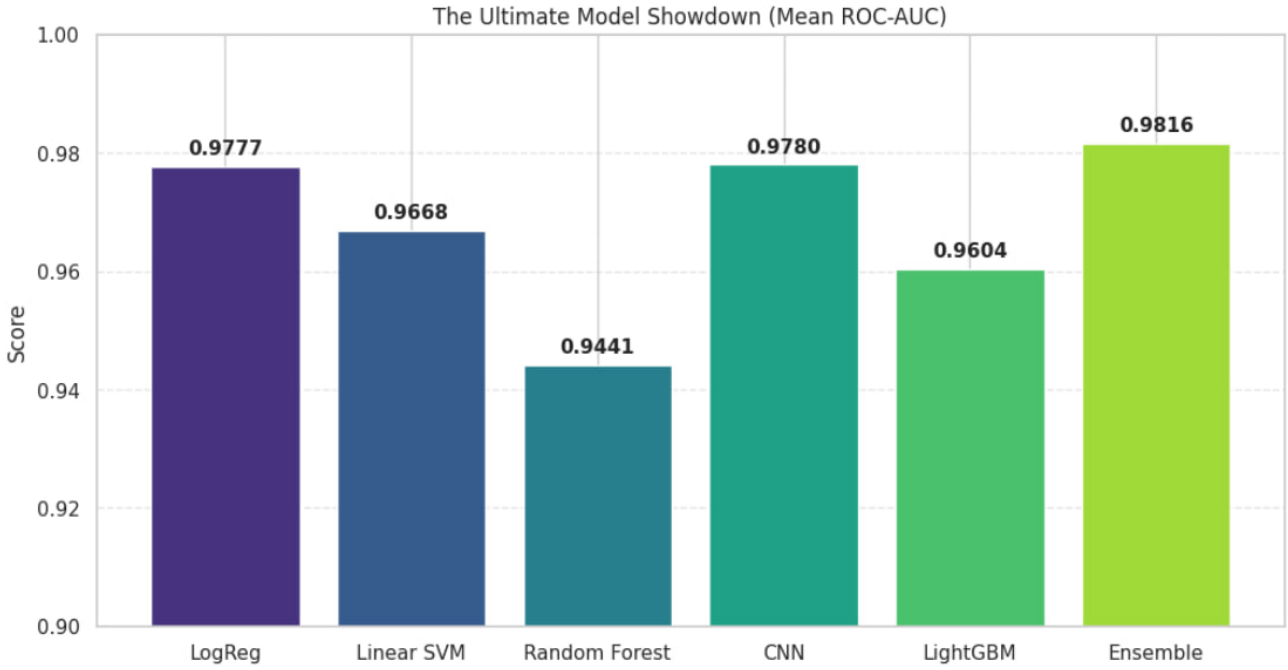

Figure 8: The Ultimate Model Showdown. The Ensemble achieves the highest performance.

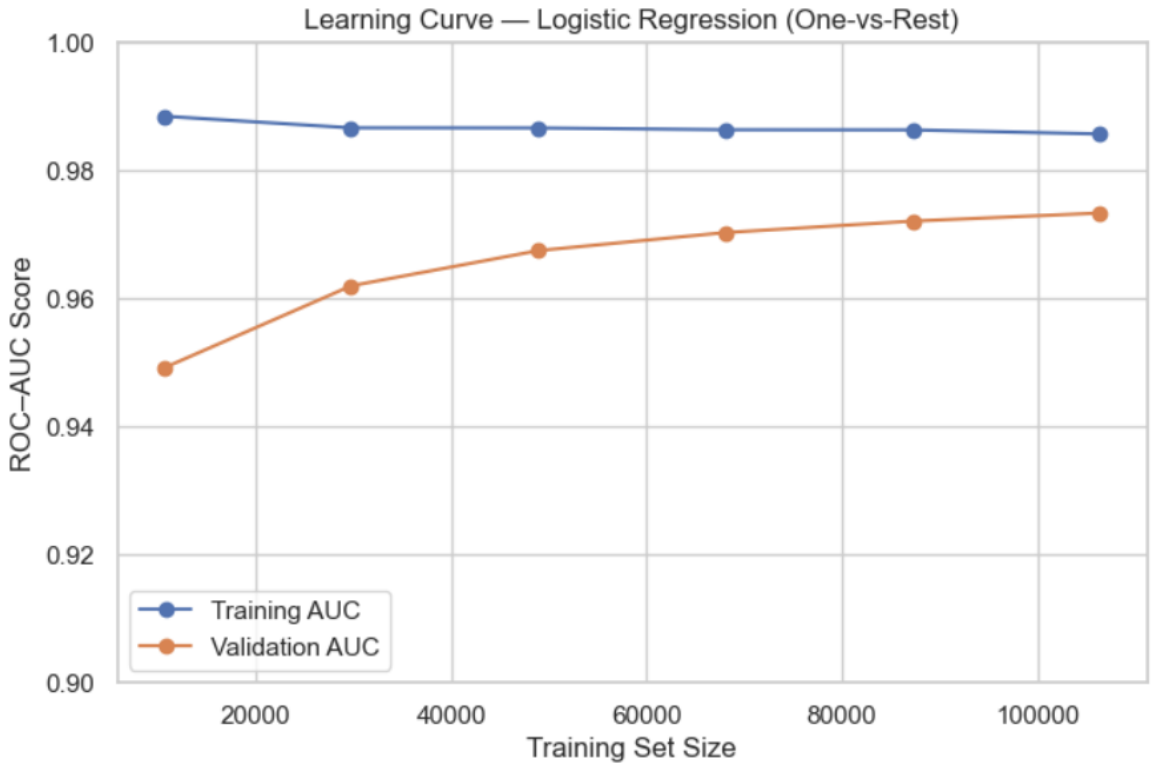

Figure 9: Learning curve for Logistic Regression.

9 Error Analysis and Optimization

9.1 Threshold Optimization

Initial analysis of our errors (using a default threshold of 0.5) revealed a critical issue common in imbalanced datasets: **High False Negatives for rare classes.** For example, for the `threat` label, the model missed 635 actual threats (False Negatives) while only flagging 15 False Positives. The model was too conservative because threats are so rare (0.3% of data).

To fix this, we performed **Threshold Tuning**. We iterated through thresholds from 0.1 to 0.9 to maximize the F1-score for each label independently.

Label	Default Thresh	Optimal Thresh	Impact
toxic	0.5	0.70	Reduced False Positives
threat	0.5	**0.15**	**Reduced False Negatives**
identity_hate	0.5	0.25	Improved detection of rare hate speech

Table 2: Impact of Threshold Optimization.

By lowering the threshold for `threat` to 0.15, we allow the model to flag potential threats even if it is only 15% sure, which is safer for content moderation. Conversely, for `toxic`, we raised the threshold to 0.70 to reduce false alarms on common profanity. Figure 10 visualizes the improved error distribution.

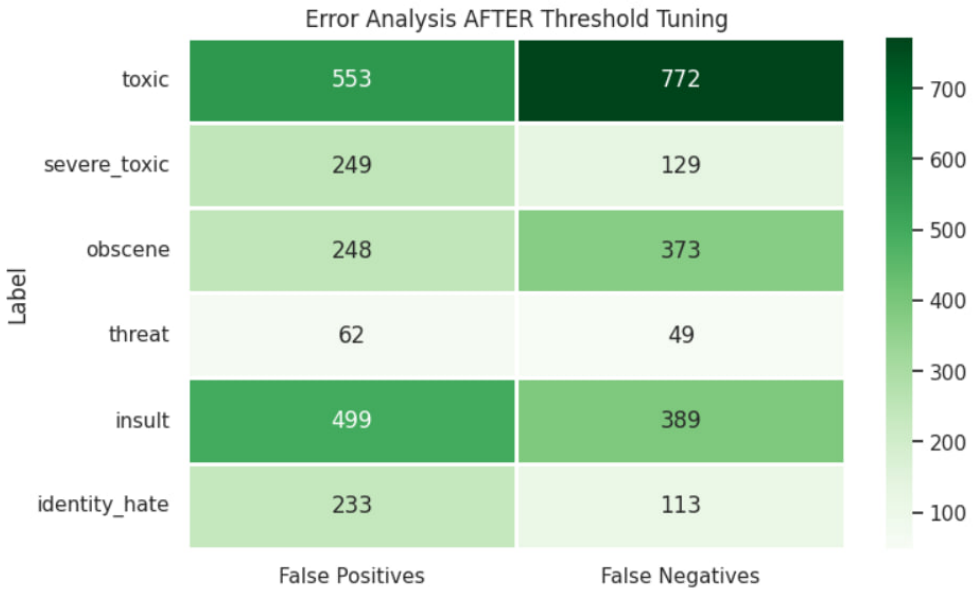

Figure 10: Error Analysis AFTER Threshold Tuning. Note the improved balance in rare classes.

10 Contributions

This project was a collaborative effort with tasks distributed to leverage individual strengths while maintaining a cohesive pipeline.

- **Reyhaneh Kharazmi Pour:** Conducted comprehensive Exploratory Data Analysis (class imbalance, length analysis), implemented parallel Preprocessing Pipelines (TF-IDF and Tokenization), and established all Baseline Models.
- **Helia Mirsoofian:** Executed 5-Fold Cross-Validation to ensure stability, generated Learning Curves, and designed the Heterogeneous Ensemble strategy that achieved our peak performance.
- **Iman Mohammadi:** Developed and tuned Advanced Models (1D-CNN and LightGBM), performed detailed Error Analysis, and implemented Threshold Optimization to fix false negatives for rare classes.

The team coordinated through regular meetings to integrate these components. For example, the decision to use a parallel preprocessing pipeline (one for TF-IDF, one for CNN) was a joint architectural decision made after initial baseline experiments showed tree-based models struggling with raw text.

11 Conclusion

In this work, we demonstrated that a heterogeneous ensemble of linear (Logistic Regression) and neural (1D-CNN) models significantly outperforms individual baselines for toxic comment classification, achieving a ROC-AUC of 0.9816. We highlighted the limitations of tree-based models on sparse text data and the necessity of threshold tuning for handling severe class imbalance. Future work could explore transformer-based architectures (e.g., BERT) to further improve semantic understanding.